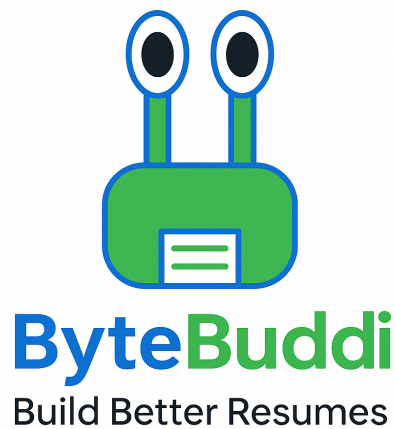


**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
THE UNIVERSITY OF TEXAS AT ARLINGTON**

**DETAILED DESIGN SPECIFICATION
CSE 4317: SENIOR DESIGN II
SPRING 2026**



**TEAM BYTEBUDDI
BYTEBUDDI**

**ADALYNN EVERLY
RYAN DAVIS
XAVIOR LOPEZ
CESAR GALLEGOS
RENE SALAZAR
ALAN ACEVEDO**

REVISION HISTORY

Revision	Date	Author(s)	Description
0.1	1.23.2026	AE, RS, XL, CG, RD, AA	document creation
0.2	2.13.2026	AE, RS, XL, CG, RD, AA	complete draft

CONTENTS

1	Introduction	5
2	System Overview	5
3	Presentation Layer Subsystems	8
3.1	Layer Hardware	8
3.2	Layer Operating System	8
3.3	Layer Software Dependencies	8
3.4	Subsystem 1-Authentication UI	8
4	Application Layer Subsystems	11
4.1	Layer Hardware	11
4.2	Layer Operating System	11
4.3	Layer Software Dependencies	11
4.4	Subsystem 1 Application Services	11
5	Data and Integration Layer Subsystems	14
5.1	Layer Hardware	14
5.2	Layer Operating System	14
5.3	Layer Software Dependencies	14
5.4	Subsystem 1 Data and Integration Services	14
6	Appendix A	17

LIST OF FIGURES

1	ByteBuddi Three-Layer Architecture Diagram	6
2	ByteBuddi Data Flow Diagram	7
3	ByteBuddi Data Flow Diagram	9
4	ByteBuddi Application Diagram	12
5	Data and Integration Layers	15

LIST OF TABLES

1 INTRODUCTION

ByteBuddi is a web-based automated resume builder designed to assist college students and entry-level job seekers in efficiently creating, editing, previewing, and exporting professional resumes tailored for internships and early-career positions. As described in the System Requirements Specification (SRS), the system allows users to manage accounts, input personal and professional information, select from multiple resume templates, and generate PDF resumes in real time. In addition to core resume-building features, ByteBuddi also supports AI-powered mock interviews and integrates with external career platforms such as Handshake to enhance career readiness and streamline job application workflows.

This document presents the detailed design of ByteBuddi and is based on the requirements defined in the System Requirements Specifications (SRS) and the system structure described in the Architectural Design Specification (ADS). The ADS outlines a modular, secure, and scalable three-layer architecture that supports key functional and non-functional requirements, including secure authentication and data storage, responsive resume rendering and export, AI-driven interview features, third-party integrations, and administrative analytics and template management. Together, these background documents provide the lead, foundation, and rationale for the design decisions detailed in the following sections.

2 SYSTEM OVERVIEW

The system is designed as a modular web-based platform that helps college students and entry-level professionals efficiently create a professional resume and practice for interviews through AI-assisted features.

The system follows a three-tier layered architecture:

- **Presentation Layer (Client):** React single-page app providing the UI for users/admins; routes include Home, Gallery, Builder, Templates, About, Login, and Interviewer.
- **Application Layer (Backend):** Node.js/Express services implementing business logic: authentication, resume management, template engine, PDF generation, AI Management, admin APIs.
- **Data and Integration Layer:** Persistent stores, object storage for assets, cache/message layer, and external APIs (AI provider, Handshake).

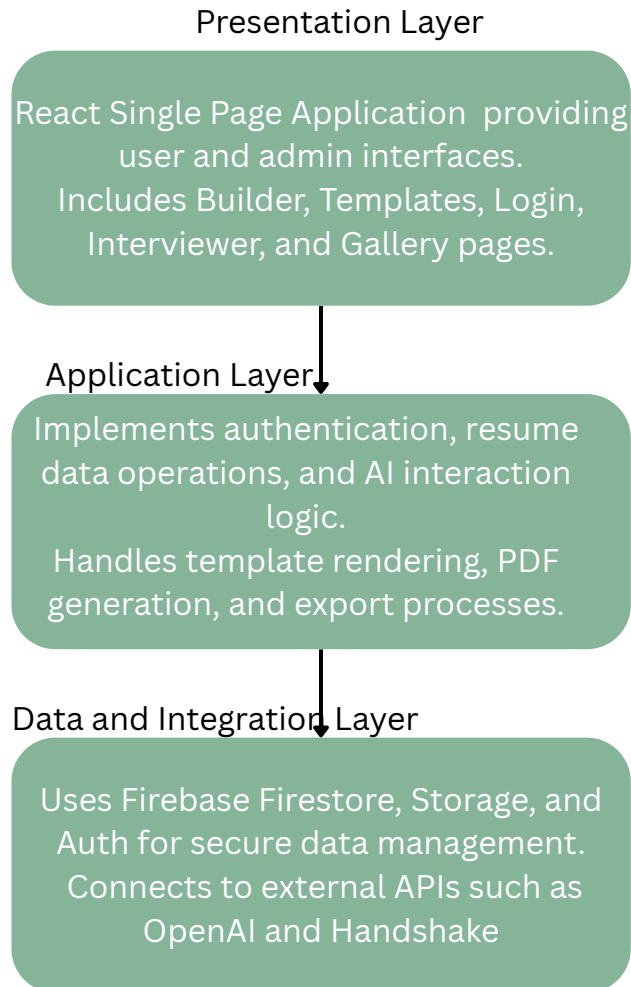


Figure 1: ByteBuddi Three-Layer Architecture Diagram

The ByteBuddi system is made up of interdependent subsystems distributed across the presentation, application, and data layers. The presentation layer handles user interaction, collection and validation of resume input, template rendering, and initiating AI interviews. The application layer, implemented through Firebase, manages authentication, synchronization with data, and communication with the external API. The data and integration layer provides connectivity through Firebase Firestore, storage, and Auth, ensuring that all information and user credentials are safely stored and retrievable. The data flows in both directions between the subsystems: user actions trigger updates to Firestore, AI requests generate the responses in the UI, and the export operations retrieve and store information in Firebase Storage, maintaining successful coordination throughout the system.

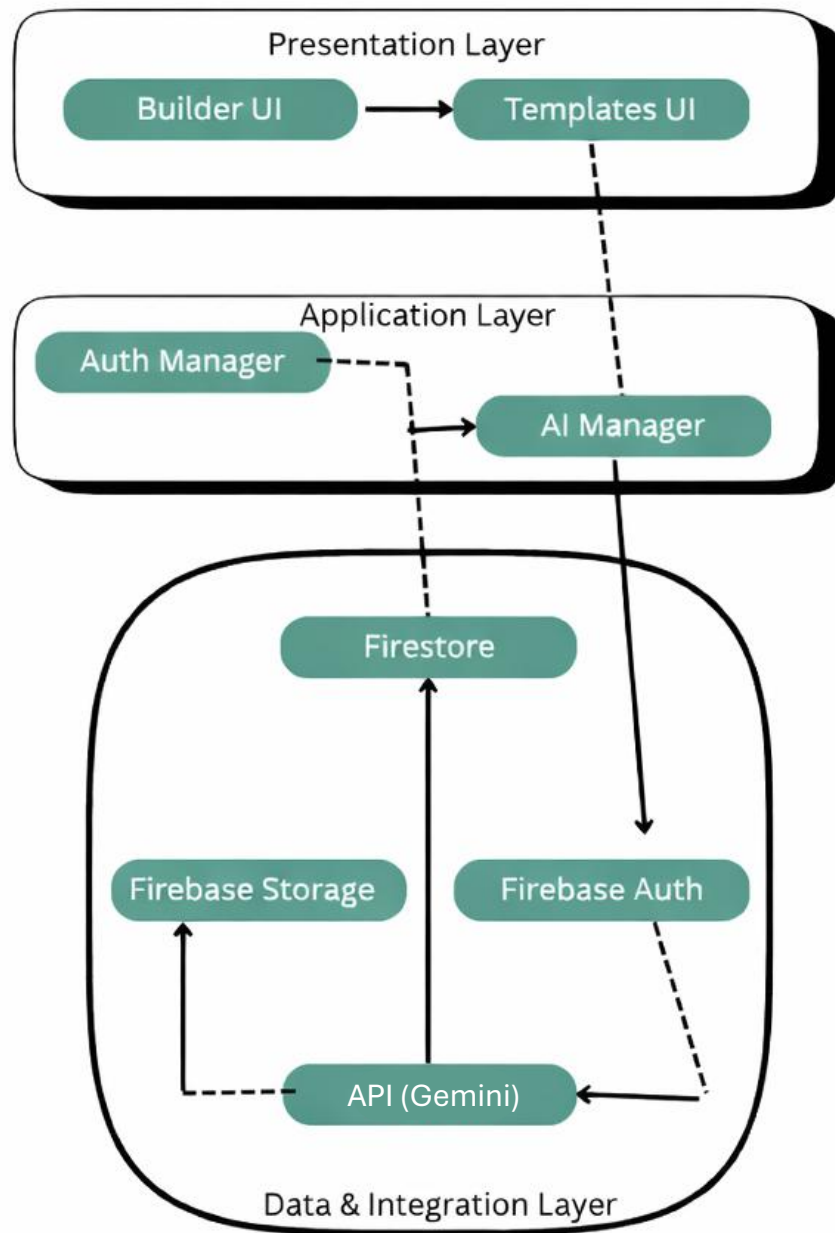


Figure 2: ByteBuddi Data Flow Diagram

3 PRESENTATION LAYER SUBSYSTEMS

In this section, the software design and runtime environment of the Presentation layer is described. The layer is defined as a web interface for the user and does not require any specific hardware components. The layer is responsible for handling all the user interactions which include: managing client-side state, rendering dynamic content and sharing data with the backend APIs and cloud storage services. The programming languages used, operating systems, and software dependencies are discussed below.

3.1 LAYER HARDWARE

The presentation layer operates exclusively as a client-side software layer executed within the browser on the user's computer or device. The input and display components such as keyboard, mouse, touch-screen, and monitor are provided by the user and are not managed by this layer.

3.2 LAYER OPERATING SYSTEM

The Presentation Layer is designed to be platform- independent and can be executed on any modern OS that can support a web browser. Including but not limited to, Windows, macOS, and Linux.

3.3 LAYER SOFTWARE DEPENDENCIES

The Presentation Layer relies on several software Frameworks and services to function correctly. The main one being the React Library, which is used to implement the single page application and manage the rendering of the UI. Additionally, dependencies also include React Router for navigation, Firebase Authentication for user authentication, Firestore for real-time data synchronization, and secure HTTPS communication for all client-server interactions.

3.4 SUBSYSTEM 1-AUTHENTICATION UI

The Authentication UI subsystem is software based in the Presentation Layer and it's responsible for managing user authentication. The primary purpose is to provide interfaces for user login and sign up while safely communicating with the backend authentication services. This subsystem specifically validates user input , displays authentication status notification, and controls who has access to the protected pages of the application bases on the user's authentication state (admin).

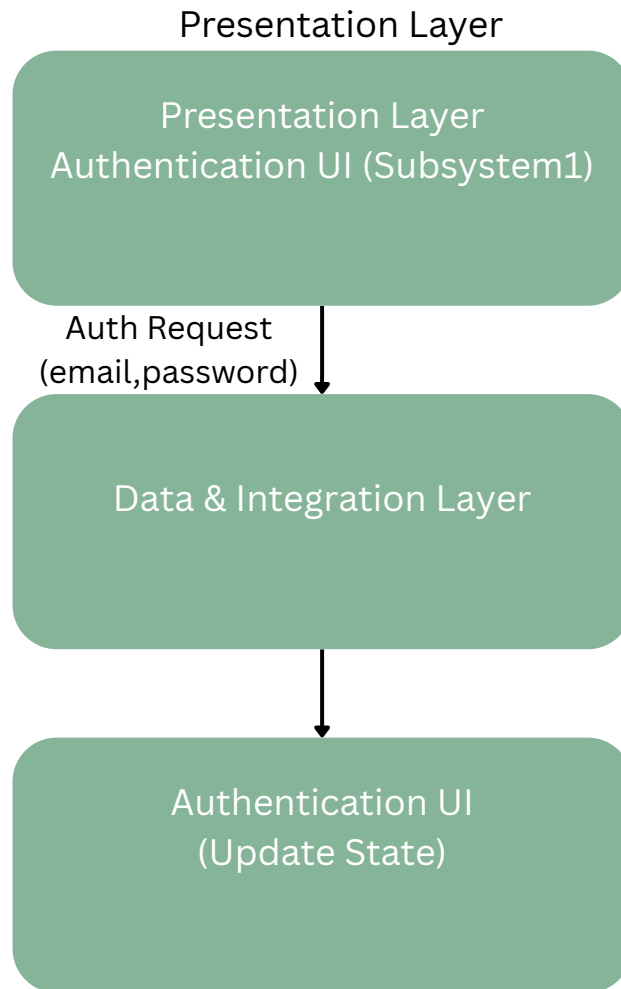


Figure 3: ByteBuddi Data Flow Diagram

3.4.1 SUBSYSTEM HARDWARE

No specific hardware is required for the Authentication UI subsystem. It executes within a web browser.

3.4.2 SUBSYSTEM OPERATING SYSTEM

The subsystem is independent and executes on any OS that supports a modern web browser.

3.4.3 SUBSYSTEM SOFTWARE DEPENDENCIES

This subsystem is dependent on the React Framework for UI rendering and Firebase Authentication services for user authentication. React Router is used to manage route access based on the user state (registered, unregistered, or admin).

3.4.4 SUBSYSTEM PROGRAMMING LANGUAGES

JavaScript and JSX are used through the React Framework.

3.4.5 SUBSYSTEM DATA STRUCTURES

State Objects are used to store authentication input, status, and error messages.

3.4.6 SUBSYSTEM DATA PROCESSING

This subsystem validates authentication before submitting requests to Firebase. It processes authentication responses to update client state , display success or error messages, and enforce route access control.

4 APPLICATION LAYER SUBSYSTEMS

In this section, the Application Layer is described in terms of its software design and runtime environment. This layer contains the backend logic that sits between the Presentation Layer and the Data and Integration Layer. Its main job is to enforce the system rules, validate requests coming from the client, and coordinate operations that involve authentication, resume management, resume export, and AI interview features.

4.1 LAYER HARDWARE

The Application Layer runs as a server side software layer hosted on cloud infrastructure. No project specific physical hardware is required because compute and networking resources are provided by the hosting environment. During development, the same services can be executed on a developer machine.

4.2 LAYER OPERATING SYSTEM

In production, the Application Layer runs in a managed server environment that is typically Linux based. For local development, the backend can be executed on Windows, macOS, or Linux as long as the required runtime and dependencies are installed.

4.3 LAYER SOFTWARE DEPENDENCIES

The Application Layer is implemented using Node.js with an HTTP service framework such as Express. It relies on Firebase services for authentication and database access. It also communicates over HTTPS to external services, including the AI provider used for mock interviews and the Handshake integration described in the system overview. If resume export is generated on the server, the layer also depends on a PDF generation library or an HTML to PDF rendering tool.

4.4 SUBSYSTEM 1 APPLICATION SERVICES

The Application Services subsystem contains the backend endpoints that the client calls during normal use of ByteBuddi. It validates incoming data, checks the user authentication state, and performs operations to create, read, and update resume content. It also coordinates resume export and mock interview requests so the client does not talk directly to external APIs

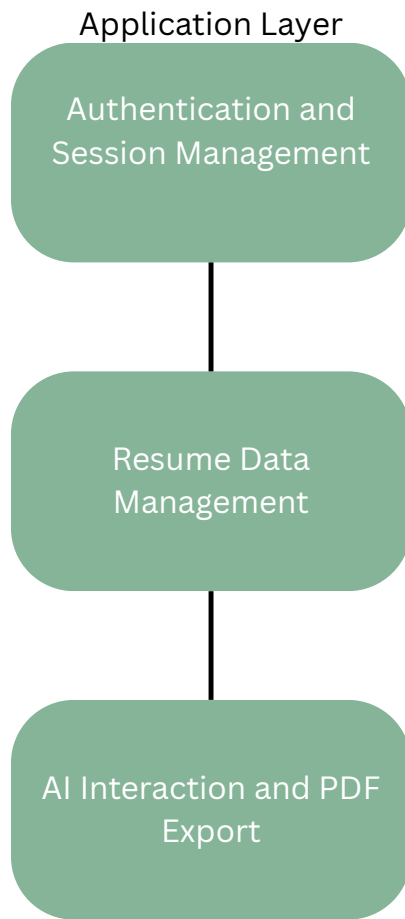


Figure 4: ByteBuddi Application Diagram

4.4.1 SUBSYSTEM HARDWARE

No dedicated hardware is required for this subsystem. It executes on cloud compute resources in production and can also run locally during development.

4.4.2 SUBSYSTEM OPERATING SYSTEM

The subsystem is intended to run in a server environment that is typically Linux based. For development, it is compatible with Windows, macOS, and Linux.

4.4.3 SUBSYSTEM SOFTWARE DEPENDENCIES

This subsystem uses the Node.js runtime and an HTTP framework for routing and middleware. It uses Firebase Authentication to verify user identity and Firestore to read and write resume data. When a user exports a resume, the subsystem uses a document rendering tool to generate a downloadable resume file. For mock interviews, the subsystem calls the AI provider and returns the results to the client. All requests are transmitted using HTTPS and responses are returned in JSON format.

4.4.4 SUBSYSTEM PROGRAMMING LANGUAGES

JavaScript is used to implement the backend services. TypeScript may also be used if typed interfaces are enabled in the project codebase.

4.4.5 SUBSYSTEM DATA STRUCTURES

The subsystem exchanges JSON objects between the client and backend. Resume data is stored and transferred as a structured object with fields for each resume section, such as education, experience, projects, and skills. Template selection is represented using a template identifier and optional configuration values. Mock interview data is stored as a session object that includes the selected focus area and the conversation history needed to continue the session.

4.4.6 SUBSYSTEM DATA PROCESSING

The processing in this subsystem is mainly validation and orchestration. When a request is received, the service first verifies the authentication token and checks that the user has permission to access the requested resource. It then validates and normalizes the input so incomplete or malformed resume data does not get stored. For create and update operations, the validated resume content is written to the database. For export operations, the stored resume content is mapped into the selected template and rendered into a downloadable document. For mock interview operations, the service builds an AI request using the selected focus area and the resume context, sends it to the AI provider, and returns the response to the client.

5 DATA AND INTEGRATION LAYER SUBSYSTEMS

In this section, the Data and Integration Layer is described in terms of storage, authentication services, and external integrations. This layer is responsible for persisting user data, storing exported resume files, and providing secure identity verification used by the rest of the system. It also supports connectivity to external services such as the AI provider for mock interviews and career platforms such as Handshake.

5.1 LAYER HARDWARE

The Data and Integration Layer is hosted on managed cloud infrastructure provided by Firebase. No project specific physical hardware is required. Storage and compute resources are provisioned and scaled by the cloud platform as needed.

5.2 LAYER OPERATING SYSTEM

In production, the Data and Integration Layer runs on managed server environments where the underlying operating system is controlled by the cloud provider and is typically Linux based. Developers do not directly manage the operating system for these services. During development, the client and backend access the same managed services through the Firebase SDKs.

5.3 LAYER SOFTWARE DEPENDENCIES

This layer relies on Firebase Firestore for persistent document storage and Firebase Storage for storing exported resume files and other assets. It uses Firebase Authentication to manage user identity and to issue tokens that are validated by the application and presentation layers. The layer also supports integration with external APIs such as OpenAI for mock interviews and Handshake for career platform connectivity. All communication with these services is performed over HTTPS using authenticated requests.

5.4 SUBSYSTEM 1 DATA AND INTEGRATION SERVICES

The Data and Integration Services subsystem contains the storage and identity components that ByteBuddi uses to keep user information safe and retrievable. It stores user profiles and resumes in Firestore so the application can load the latest resume state on demand. It stores exported resume documents in Firebase Storage so users can download the generated output. It also provides authentication services through Firebase Auth so only signed in users can access protected pages and only authorized users can access admin functionality.

5.4.1 SUBSYSTEM HARDWARE

No dedicated hardware is required for this subsystem. It is provided as managed cloud services through Firebase.

5.4.2 SUBSYSTEM OPERATING SYSTEM

The subsystem runs in a managed cloud environment where the operating system is handled by the provider. The project interacts with it through SDKs and HTTPS requests rather than direct system access.

5.4.3 SUBSYSTEM SOFTWARE DEPENDENCIES

This subsystem depends on Firebase Firestore, Firebase Storage, and Firebase Authentication. It also depends on Firebase security rules and access control configuration to restrict reads and writes. The client side and server side components use Firebase SDK libraries to authenticate users and to read and write data. For external integrations, the backend uses an API client to communicate with OpenAI and Handshake and stores any required tokens or references in Firestore when needed.

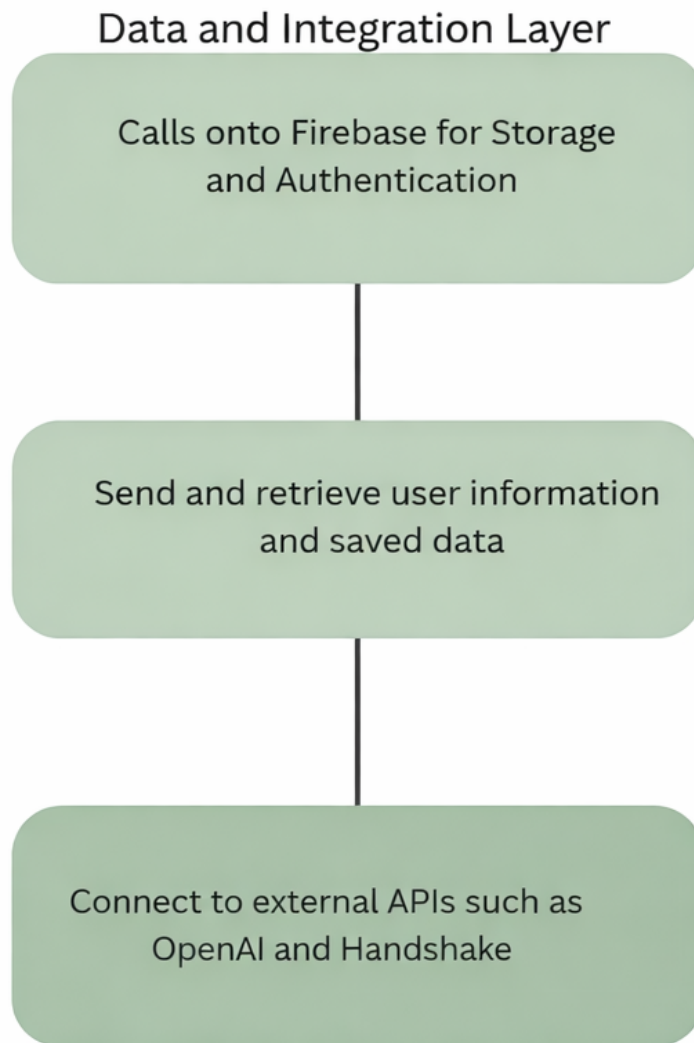


Figure 5: Data and Integration Layers

5.4.4 SUBSYSTEM PROGRAMMING LANGUAGES

This subsystem is primarily service based and does not require a dedicated programming language inside the platform itself. The code that interacts with it is written in JavaScript through the Firebase SDK in the client and backend layers.

5.4.5 SUBSYSTEM DATA STRUCTURES

Data is stored in Firestore as structured documents. A user profile document stores the user identifier and basic account information. Resume documents store the resume content in a section based structure that includes education, experience, projects, and skills. Template related configuration can be stored as template documents or as fields within a resume document depending on how the application organizes templates. Exported resumes are stored in Firebase Storage and referenced by metadata in Firestore, such as the storage path, export timestamp, and the template identifier used for the export. Authentication state is represented by Firebase Auth user records and signed tokens that are used to control access throughout the system.

5.4.6 SUBSYSTEM DATA PROCESSING

The primary processing in this subsystem is secure storage and retrieval. When a user signs in, Firebase Auth issues an identity token that the client uses for authenticated requests. When the user creates or edits a resume, the application writes the updated resume document to Firestore and the client can read the latest version when the page reloads. When the user exports a resume, the generated document is uploaded to Firebase Storage and the corresponding metadata is stored in Firestore so the system can track and retrieve the export. Security rules enforce that users can only access their own resume data and that only authorized admin accounts can access admin resources. External integrations are routed through the backend so API keys and sensitive tokens are not exposed to the client, and only the necessary results are returned back to the UI.

6 APPENDIX A

REFERENCES

Architectural Design specification Latex document.
System Requirement Specification Latex document.